

Introduction to R

Session 04: Tidy Data

Álvaro Pérez

Instituto Tecnológico Autónomo de México

Spring 2025

Overview of Today's Session

Tidy Data

Pivot Functions

Tidy Verbs

Missing Values

Three Rules

- ▶ Each **variable** must have its own column
- ▶ Each **observation** must have its own row
- ▶ Each **value** must have its own cell

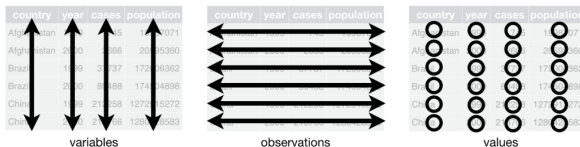


Figure: Three rules

This table has three observations per row and the total_meanRT column contains two values.

Table 4.1: Untidy table

id	score_1	score_2	score_3	rt_1	rt_2	rt_3	total_meanRT
1	4	3	7	857	890	859	14 (869)
2	3	1	1	902	900	959	5 (920)
3	2	5	4	757	823	901	11 (827)
4	6	2	6	844	788	624	14 (752)
5	1	7	2	659	764	690	10 (704)

This is the tidy version.

Table 4.1: Tidy table

id	trial	rt	score	total	mean_rt
1	1	857	4	14	869
1	2	890	3	14	869
1	3	859	7	14	869
2	1	902	3	5	920
2	2	900	1	5	920
2	3	959	1	5	920
3	1	757	2	11	827
3	2	823	5	11	827
3	3	901	4	11	827
4	1	844	6	14	752
4	2	788	2	14	752
4	3	624	6	14	752

The pivot functions allow you to transform a data table from wide to long or long to wide in one step.

- ▶ **pivot_longer()** converts a wide data table to long format by converting the headers from specified columns into the values of new columns, and combining the values of those columns into a new condensed column.
 - ▶ `cols` refers to the columns you want to make long. You can refer to them by their names, like `col1`, `col2`, `col3`, `col4` or `col1:col4` or by their numbers, like `8`, `9`, `10` or `8:10`.
 - ▶ `names_to` is what you want to call the new columns that the gathered column headers will go into; it's "domain" and "qnumber" in this example.
 - ▶ `names_sep` is an optional argument if you have more than one value for `names_to`. It specifies the characters or position to split the values of the `cols` headers.
 - ▶ `values_to` is what you want to call the values in the columns they're "score" in this example.

Longer

```
1 table4a
2
3 table4a %>%
4   pivot_longer(c(`1999`, `2000`), names_to = "year",
                values_to = "cases")
```

country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

table4

Figure: Pivoting into a longer tidy form

Pivoting

Longer

In the final result, the pivoted columns are dropped, and we get new year and cases columns.

`pivot_longer()` makes datasets longer by increasing the number of rows and decreasing the number of columns.

We can use `pivot_longer()` to tidy `table4b` in a similar fashion. The only difference is the variable stored in the cell values:

```
1 table4b %>%  
2   pivot_longer(c(`1999`, `2000`), names_to = "year",  
               values_to = "population")
```


We can also go from long to wide format using the **pivot_wider()** function.

- ▶ `names_from` is the columns that contain your new column headers.
- ▶ `values_from` is the column that contains the values for the new columns.
- ▶ `names_sep` is the character string used to join names if `names_from` is more than one column.

```
1 table2
```

To tidy this up, we first analyse the representation in similar way to `pivot_longer()`. This time, however, we only need two parameters:

- ▶ The column to take variable names from. Here, it's `type`.
- ▶ The column to take values from. Here it's `count`.

Once we've figured that out, we can use `pivot_wider()`

```
1 table2 %>%  
2   pivot_wider(names_from = type, values_from = count  
               )
```



country	year	key	value
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Figure: Pivoting into a wider tidy form

The pivot functions above are relatively new functions that combine the four basic tidy verbs.

You can also convert data between long and wide formats using these functions.

Tidy Verbs

gather()

Much like `pivot_longer()`, **`gather()`** makes a wide data table long by creating a column for the headers and a column for the values. The main difference is that you cannot turn the headers into more than one column.

```
1 personality_gathered <- gather(  
2   data = personality,  
3   key = "question", # new column name for gathered  
4     headers  
5   value = "score",  # new column name for gathered  
6     values  
7   Op1:Ex9          # columns to gather  
8 ) %>%  
9 glimpse()
```

separate()

`separate()` pulls apart one column into multiple columns, by splitting wherever a separator character appears. Take `table3`:

```
1 table3
```

The `rate` column contains both cases and population variables, and we need to split it into two variables. `separate()` takes the name of the column to separate, and the names of the columns to separate into

```
1 table3 %>%  
2   separate(rate, into = c("cases", "population"))
```

separate()

By default, `separate()` will split values wherever it sees a non-alphanumeric character (i.e. a character that isn't a number or letter). We could rewrite the previous code as:

```
1 table3 %>%  
2   separate(rate, into = c("cases", "population"), sep  
   = "/" )
```

country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Figure: Separating makes it tidy

Tidy Verbs

unite()

`unite()` is the inverse of `separate()`: it combines multiple columns into a single column. You'll need it much less frequently than `separate()`, but it's still a useful tool to have in your back pocket.

- ▶ `col` is your new united column
- ▶ `...` refers to the columns you want to unite
- ▶ `sep` is the character(s) that will separate your united columns



country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

country	century	year	rate
Afghanistan	19	99	745 / 19987071
Afghanistan	20	0	2666 / 20595360
Brazil	19	99	37737 / 172006362
Brazil	20	0	80488 / 174504898
China	19	99	212258 / 1272915272
China	20	0	213766 / 1280428583

Figure: Uniting makes it tidy

spread()

You can convert data from long format into wide format.

```
1 personality_spread <- spread(  
2   data = personality_unite,  
3   key = domain_n, # column that contains new headers  
4   value = score    # column that contains new values  
5 ) %>%  
6   glimpse()
```

Changing the representation of a dataset brings up an important subtlety of missing values. Surprisingly, a value can be missing in one of two possible ways:

- ▶ Explicitly, i.e. flagged with NA.
- ▶ Implicitly, i.e. simply not present in the data.

```
1 stocks <- tibble(  
2   year      = c(2015, 2015, 2015, 2015, 2016, 2016, 2016)  
3   ,  
4   qtr       = c( 1,    2,    3,    4,    2,    3,    4)  
5   ,  
6   return    = c(1.88, 0.59, 0.35,  NA, 0.92, 0.17, 2.66)  
7 )
```

Because these explicit missing values may not be important in other representations of the data, you can set `values_drop_na = TRUE` in `pivot_longer()` to turn explicit missing values implicit:

```
1 stocks %>%  
2   pivot_wider(names_from = year, values_from = return)  
3   %>%  
4   pivot_longer(  
5     cols = c(`2015`, `2016`),  
6     names_to = "year",  
7     values_to = "return",  
8     values_drop_na = TRUE  
9   )
```